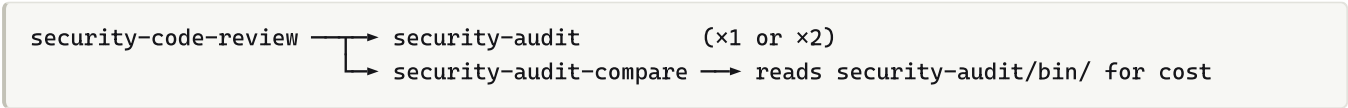


Skills Reference

Three skills. Each is a folder of Markdown and static assets, installs by copying, and has no runtime dependency.

Skill	One line	Invoked by
<code>security-code-review</code>	Runs the whole review: asks, validates, sequences, announces	The user
<code>security-audit</code>	Audits one source tree → one HTML report	The orchestrator, or directly
<code>security-audit-compare</code>	Joins two audits → the comparison report	The orchestrator, or directly



Keep the three as siblings in the same skills directory.

1. `security-code-review`

The entry point. The only one a user normally types.

Invocation

```
/security-code-review [options]
```

Option	Default	Meaning
<code>--name "<Project>"</code>	<i>asked</i>	Project name; also the output folder name
<code>--legacy <path></code>	<i>asked</i>	Legacy source tree. Empty / <code>none</code> / <code>skip</code> = no legacy phase
<code>--modernized <path></code>	<i>asked</i>	Modernized source tree. Required
<code>--out <dir></code>	current directory	Parent directory; the project folder is created inside
<code>--depth quick standard deep</code>	<code>standard</code>	Passed to both audits
<code>--yes</code>	off	Skip the confirmation step (UI and CI callers)

Anything given on the command line is not asked for again — the same skill serves an engineer typing in VS Code and a UI shelling out with every argument supplied.

What it does

1. **Collects** project name, legacy path, modernized path — in one message, then waits.
2. **Validates** before spending anything: paths resolve and exist, the name is filename-legal, the two paths are not the same tree, and file counts look plausible.
3. **Confirms** the plan, showing resolved absolute paths, file counts and the output location.

4. **Creates** `<out>/<Project name>/` with its `.security-audit/` working directory and `run.json`.
5. **Runs the phases sequentially** — Legacy (if given), Modernized, Comparison (if both succeeded).
6. **Announces** scores, bucket counts, report paths and the total cost.

Phase configuration

	Phase A	Phase B	Phase C
Skill	<code>security-audit</code>	<code>security-audit</code>	<code>security-audit-compare</code>
Flags	<code>--variant legacy --mode analyze</code>	<code>--variant modernized --mode plan</code>	—
Remediation	none	full	carried from Phase B
Runs when	legacy path given	always	both audits succeeded

Behaviour worth knowing

- **No legacy path** → no Legacy report **and** no Comparison report. The run produces the Modernized report only, and says so. It never synthesizes a comparison against an imagined baseline.
- **A failed phase does not abort the run.** If Phase A fails, Phase B still delivers; the comparison is skipped and the reason stated in both `run.json` and the summary.
- **Existing project folder** → it asks whether to overwrite or use a dated subfolder. It never silently overwrites reports someone may have circulated.
- **Both paths identical** → it stops and asks. Comparing a tree with itself yields an all-inherited report and a zero delta.

Files

```
security-code-review/
└─ SKILL.md          the whole skill - orchestration is procedure, not assets
```

2. `security-audit`

The audit engine. Audits exactly one source tree. Knows nothing about migrations or the other side — which is what lets it be used on its own for any codebase.

Invocation

```
/security-audit <target-path> [options]
```

Option	Default	Meaning
<code><target-path></code>	<i>required</i>	Directory or file to audit. Hard scope boundary
<code>--name "<Project>"</code>	inferred	Project name for the title and filename

Option	Default	Meaning
<code>--out <dir></code>	<code><target>/../security-reports</code>	Where the report is written
<code>--depth quick standard deep</code>	<code>standard</code>	Effort tier
<code>--mode analyze plan</code>	<code>plan</code>	<code>analyze</code> = findings only, no remediation
<code>--variant legacy modernized standalone</code>	<code>standalone</code>	Labels the report and its filename
<code>--exclude "<globs>"</code>	see below	Extra comma-separated globs to skip
<code>--baseline <findings.json></code>	none	Prior run to diff against (New / Persisting / Resolved)
<code>--no-cost</code>	off	Skip cost accounting

Modes

	<code>plan</code> (default)	<code>analyze</code>
Findings, evidence, severity, confidence	yes	yes
Standards mapping, strengths, evidence log	yes	yes
Current score	yes	yes
Per-finding <code>remediation</code> + <code>verification</code>	yes	omitted entirely
<code>roadmap</code> and projected score	yes	omitted entirely

`analyze` is diagnosis-only, for codebases nobody intends to fix — a legacy system being retired is the canonical case. The omission is stated in the report's Limitations so a reader never wonders whether the fixes were forgotten.

Depth tiers

Tier	Rule packs	Manual review	Relative cost
<code>quick</code>	universal + detected stack	Critical/High evidence only	1×
<code>standard</code>	all matching	every hit triaged; auth and data-flow read of entry points	~3×
<code>deep</code>	all matching	above, plus every entry point, trust boundary and data sink; business logic	~8×

The nine steps

0	Cost watermark	start the transcript segment
1	Inventory & stack	Glob manifests, count files/lines, identify entry points
2	Deterministic pass	~109 ripgrep rules via the Grep tool → candidates
3	Triage	read the code, reachability, confidence, dedupe, + design flaws grep cannot see ← the part that matters
4	Dependency check	manifests; OSV/advisory only with consent
5	Score	weighted categories, 0-100, grade, posture band
6	Remediation plan	non-breaking fixes + phased roadmap (plan mode only)
7	Collect cost	diff the transcript from the watermark
8	Render	part1.html + findings.json + part2.html
9	Report back	score, counts, top three fixes, path, cost

The rule packs

Data, not code — `rules/*.json`, executed through the `Grep` tool.

Pack	Covers
<code>universal.json</code>	Secrets, weak crypto, dangerous primitives, PII in logs, TLS bypass
<code>dotnet.json</code>	.NET / ASP.NET Core / Blazor / Razor / EF Core
<code>java.json</code>	Java / Kotlin / Spring / Jakarta / Hibernate
<code>javascript.json</code>	Node, Express, React / Angular / Vue
<code>python.json</code>	Python, Django, Flask, FastAPI, SQLAlchemy
<code>php-go-ruby.json</code>	PHP/Laravel, Go, Ruby/Rails
<code>sql-legacy.json</code>	Stored procedures, COBOL, Informix 4GL, VB6, Delphi, RPG
<code>config-iac.json</code>	Docker, Terraform, CI, appsettings, web.config, <code>.env</code>

Each rule carries `expect`:

- `absent-is-ok` — hits are candidate findings.
- `present-required` — a **zero-hit** result is the candidate (no `[Authorize]` anywhere, no CSP header).
These absence checks catch what pattern matching structurally cannot.

Rules are a **sensor, not a verdict**. Nothing reaches the report untriaged.

Files

```
security-audit/
├── SKILL.md
├── rules/00-index.json          pack registry + stack matching
├── rules/*.json                eight rule packs
├── references/triage-protocol.md turning grep hits into findings
├── references/scoring.md       the scoring model and its arithmetic
├── references/standards-mapping.md OWASP / CWE / ATT&CK / NIST crosswalk
├── references/remediation-playbook.md non-breaking fix patterns
├── references/report-schema.md the findings.json contract
├── assets/report.part1.html    template head (stylesheet)
├── assets/report.part2.html    template tail (the renderer)
├── assets/pricing.json         model token rates
├── assets/python-bootstrap.json pinned CPython + SHA-256 for the opt-in bootstrap
├── bin/ensure_python.sh|.ps1   resolve an interpreter; never demands an install
├── bin/cost.py                 cost collector (preferred)
├── bin/collect-cost.sh         cost collector (bash + awk)
├── bin/collect-cost.ps1        cost collector (PowerShell 5.1)
├── bin/cost.awk                transcript usage extractor
├── tests/run_corpus.py         rule regression tests (maintainers only)
└── examples/                  a worked findings.json and its report
```

Maintaining the rule packs

Any change to `rules/*.json` must be followed by `python tests/run_corpus.py`. It checks that every pattern compiles under ripgrep's Rust regex engine (no lookahead, no backreferences) and that rules fire on the vulnerable fixtures **and stay silent on the safe ones**. The silence half is the one that matters: a rule that flags parameterized SQL teaches engineers to ignore the report, and that loss is permanent.

This is the only part of the system that needs Python, and only for maintainers — never for users.

3. `security-audit-compare`

The comparison engine. Consumes two `findings.json` files and produces the comparison report.

Invocation

```
/security-audit-compare --legacy <findings.json> --modernized <findings.json> [options]
```

Option	Default	Meaning
<code>--legacy <path></code>	<i>required</i>	The legacy audit's <code>findings.json</code>
<code>--modernized <path></code>	<i>required</i>	The modernized audit's <code>findings.json</code>
<code>--name "<Project>"</code>	from the audits	Project name for the title and filename
<code>--out <dir></code>	the modernized report's directory	Where the HTML report is written
<code>--cost <dir>[,<dir>]</code>	none	Phase <code>cost.json</code> directories to roll into the cost section
<code>--no-cost</code>	off	Skip cost accounting

Both inputs are required. Running against one audit and inferring the other side is refused — the entire output is a claim about two codebases, and half the evidence produces confident fiction.

Preconditions, checked before any work

- 1. Both files exist and parse as JSON.
- 2. Both are `security-audit` outputs (carry `meta`, `scores`, `findings`).
- 3. **Same** `scores.profile` on both sides — otherwise category numbers are not comparable, and it stops rather than rescaling by hand.
- 4. The two audits are plausibly of related systems.

The five buckets

Bucket	Meaning
<code>inherited</code>	Present before and after. The migration carried the flaw across
<code>introduced</code>	New in the modernized tree, on ground the legacy system occupied and got right. A regression
<code>new-surface</code>	New in the modernized tree, on ground that did not exist before. The cost of the new architecture, not a regression
<code>resolved</code>	In legacy, gone in modernized — then interrogated for <i>why</i>
<code>not-comparable</code>	No counterpart existed, or the other side never assessed that category

The `resolved` interrogation

Every resolved item gets a `resolution.reason`:

Reason	Report treatment
<code>fixed-by-construction</code>	A real win — the new framework makes the flaw impossible
<code>explicitly-remediated</code>	A real win — someone deliberately addressed it
<code>feature-not-migrated</code>	Not a win. Raised as a coverage risk in its own section
<code>not-applicable</code>	Neutral — the weakness class cannot arise in the target architecture

This is the distinction a reader most needs and is least likely to make unaided: "12 resolved" reads as twelve fixes when some are unbuilt features whose vulnerabilities return with them.

Matching rules, in brief

- Join key is **weakness class × functional area**, never file path.
- The functional area map is built **first**, before any classification.
- Every pairing publishes a confidence: `Certain` / `Probable` / `Tentative`. Weaker than `Tentative` is not a match — both findings are left unpaired.
- Before anything is called `introduced`, the legacy audit must actually have **assessed** that category. No assessment means no evidence the old system was clean.
- Scores are **never recomputed** — each audit owns its numbers; the comparison joins them.

Full procedure: `references/matching-protocol.md`.

Comparison depth

Depth	When	What the report may claim
<code>full</code>	Both audits <code>standard</code> +, area map solid	Everything
<code>partial</code>	One audit <code>quick</code> , or gaps in the map	Bucket what is mappable; rest is <code>not-comparable</code>
<code>category-only</code>	Area map could not be built	Category scores and counts only — no per-finding buckets

Depth is stated in the report, not hidden. A `partial` comparison that names its gaps is more useful than a `full` one that invented the map.

Files

<code>security-audit-compare/</code>	
<code>— SKILL.md</code>	
<code>— references/matching-protocol.md</code>	how findings are paired and bucketed
<code>— references/comparison-schema.md</code>	the <code>comparison.json</code> contract
<code>— assets/compare.part1.html</code>	template head (shared stylesheet + comparison styles)
<code>— assets/compare.part2.html</code>	template tail (the renderer)
<code>— examples/comparison.example.json</code>	a worked <code>comparison.json</code> + its rendered report

Cost collection reuses `../security-audit/bin/` and `../security-audit/assets/pricing.json` rather than shipping a second copy. Two pricing tables drift, and the moment they do the per-phase figures here stop reconciling against the per-run figures in the other two reports. If `security-audit` is absent, this skill still renders its report with cost marked not collected.

Shared conventions

Report filenames

<code><Project></code>	- Security analysis report. - YYYY-MM-DD.html	(standalone audit)
<code><Project></code>	- Legacy - Security analysis report. - YYYY-MM-DD.html	
<code><Project></code>	- Modernized - Security analysis report. - YYYY-MM-DD.html	
<code><Project></code>	- Comparison - Security analysis report. - YYYY-MM-DD.html	

The period after "report" is part of the format. Only characters illegal in filenames

<code>(\/:*?" < ></code>	<code>)</code> are stripped from the project name.
--------------------------------	--

The render contract

Every report is `part1.html` + a JSON file + `part2.html`, concatenated.

<code>cat part1.html findings.json part2.html > "<report>.html"</code>

```
Get-Content part1.html,findings.json,part2.html -Raw | Set-Content "<report>.html" -Encoding utf8
```

The model authors data and never writes report HTML. The template renders it. This keeps output consistent across runs, removes an entire class of escaping bugs, and spends tokens on analysis rather than markup. It also means a malformed JSON file produces a visible error card rather than a silently wrong report.

Scope confinement

Every audit resolves its target to an absolute **ROOT** and never reads outside it — no `../`, ever. The only paths outside ROOT that are touched are the skill's own files, the output directory, and the session transcript (for cost only). All finding paths are recorded relative to ROOT, so reports are portable and do not leak the auditor's directory layout.

Network and MCP

The core audit requires neither. Static analysis, triage, scoring, rendering and cost accounting are all local. Two things are *improved* by network access, and the report always states which was used:

Need	Preferred	Fallback	If neither
Confirm CVEs for dependencies	An advisory MCP server	<code>WebFetch</code> against OSV	Report the inventory + a finding for missing dependency scanning
Confirm current hardening guidance	A vendor-docs MCP server	<code>WebFetch</code> of vendor docs	Use <code>references/</code> , labelled with its version date

When a run had neither, the Limitations section must say so explicitly. Silence there would be the most misleading thing the system could do.